

PAULT DOCUMENTATION

Ux Audit

Source: docs/UX_AUDIT.md

Theme: Clean Technical

Generated: February 26, 2026

Pault — UX Audit

Version: Phase 2.5 (post-settings expansion, about screen, menu bar enhancement)

Date: 2026-02-18

Methodology: User flow mapping + Nielsen's 10 Heuristics + macOS Human Interface Guidelines

Surfaces covered: Main Window · Menu Bar Popover · Hotkey Launcher · Settings · About · Onboarding

Severity Scale

| Level | Meaning |

|-----|-----|

| **Critical** | Blocks a core task, causes silent data loss, or produces broken/misleading UI |

| **Major** | Degrades a recurring experience in a noticeable way; likely to frustrate or confuse users |

| **Minor** | Polish issue, inconsistency, or missed platform convention; does not block tasks |

Section 1: User Flow Inventory

Each flow is documented with its trigger, step-by-step interaction path, and all edge cases or dead ends the user may encounter.

Flow 1 — First Launch & Onboarding

Surface: Main Window

Trigger: App launched for the first time (`hasCompletedOnboarding == false`)

Steps:

1. App opens to main window (empty sidebar)
2. Onboarding sheet appears immediately over the main window
3. Page 1 of 3: Welcome — icon, title, subtitle, one-sentence description
4. User taps "Next" → Page 2: Three Ways to Access (menu bar, main window, `Cmd+Shift+P`)
5. User taps "Next" → Page 3: Built for Speed (keyboard shortcuts)
6. User taps "Get Started" → sheet dismisses, `hasCompletedOnboarding = true`
7. Main window is now visible with empty sidebar and empty detail pane

Edge cases:

- User clicks "Back" on page 2 or 3 → returns to previous page

- User closes the sheet via Esc or window close button → `hasCompletedOnboarding` remains `false`; onboarding re-appears on next launch
- No way to re-trigger onboarding from within the app after completion
- Onboarding mentions `Cmd+1-9` and "paste instantly" — paste was removed in Phase 2.5; description is stale

Flow 2 — Create a Prompt (Main Window)

Surface: Main Window

Trigger: `Cmd+N` keyboard shortcut, or "+" toolbar button

Steps:

1. New Prompt window opens (700×620, separate window)
2. User types a title in the title field
3. User types content in the `TextEditor`
4. (Optional) User types `{{variable}}` syntax in content → variable badges appear below content automatically
5. (Optional) User clicks "+" to add tags → `TagPickerPopover` opens (260×380)
 - Existing tags shown; click to select
 - Or type name + pick color → "Create" button → tag added and selected
6. User presses `Cmd+■` or clicks "Create Prompt"
7. Window closes, main window selects the new prompt in the sidebar

Edge cases:

- "Create Prompt" disabled if both title AND content are empty (either alone is sufficient)
- Whitespace-only title or content treated as empty for validation
- No tag created if tag name field is empty in popover
- No feedback if prompt creation fails (service errors logged silently)
- If user closes window via red button or `Cmd+W`, prompt is discarded with no confirmation
- Variable badges are display-only in creation — values are filled after creation in preview/edit

Flow 3 — Create a Prompt (Menu Bar)

Surface: Menu Bar Popover

Trigger: Click "New" (+ icon) in bottom bar of popover

Steps:

1. Inline sheet slides up within the popover
2. User types title in bordered text field
3. User types content in `TextEditor` (120px height)
4. User presses `Cmd+■` or clicks "Create"
5. Sheet dismisses, new prompt appears in the prompt list

Edge cases:

- "Create" disabled if both fields are empty
- No tag support in menu bar creation flow
- No template variable detection or display during creation
- User can press Esc or Cmd+. to cancel; no confirmation needed (nothing saved)
- No way to add attachments from this surface

Flow 4 — Browse & Select a Prompt

Surface: Main Window

Trigger: App is open with at least one prompt in the library

Steps:

1. Sidebar shows prompt list under active filter ("All Prompts" by default)
2. User scrolls or clicks a filter ("Recently Used", "Archived", or a tag pill)
3. User clicks a prompt row → detail pane shows read-only preview
4. Detail pane shows: title, content (rich text, read-only), variable fill fields (if applicable), attachments strip, inspector toggle

Edge cases:

- Clicking a tag pill in a sidebar row changes filter to `.tag( tag )` — not obvious to first-time users
- If filter is "Recently Used" and no prompts have `lastUsedAt` set, list is empty with message "No recently used prompts" — no hint that copying a prompt populates this
- Tags beyond 2 are hidden with "+N" — user cannot see all tags without opening inspector
- Double-clicking a row opens the Edit window — not communicated anywhere in the UI

Flow 5 — Fill Template Variables & Copy

Surface: Main Window (primary), Menu Bar / Hotkey Launcher (copy only)

Trigger: User selects a prompt that contains `{{variable}}` syntax

Steps:

1. Detail pane shows content (read-only, max 200px height when variables present)
2. Below content: "Fill in template variables" label
3. `InlineVariablePreview` shows editable text fields for each variable in order
4. Variables summary panel shows all variables with their fill status (e.g. "2/3 filled")
5. As user types in fields → resolved preview updates live below
6. User presses Cmd+C or clicks "Copy" toolbar button
7. Clipboard receives resolved content (unfilled variables remain as `{{name}}`)
8. Toast: "Copied to clipboard" appears briefly

Edge cases:

- Tab/Shift+Tab cycles through variable fields
- If no variables are filled, copy still works — unfilled `{{variables}}` appear in clipboard output; no warning
- Multiple occurrences of the same variable name are tracked independently (e.g. `{{name}}` [1], `{{name}}` [2])
- Variable values persist across sessions (stored in `SwiftData`)
- Menu bar and hotkey launcher copy with whatever values are currently stored — no in-context editing
- Rich text (RTFD) content copied to clipboard alongside plain text when applicable
- `lastUsedAt` is updated on copy, so prompt appears in "Recently Used"

Flow 6 — Edit an Existing Prompt**Surface:** Main Window (Edit Window)**Trigger:** Double-click sidebar row, `Cmd+E` shortcut, or "Edit" button (pencil) in toolbar**Steps:**

1. Edit window opens (700×620 min size, separate window from main window)
2. Title field is editable (debounced save, 500ms)
3. `RichTextEditor` is editable — supports rich text, drag-drop images, inline attachments
4. Template variables sync automatically as content changes (300ms debounce)
5. Variable fill fields appear in panel below (same tab navigation)
6. Inspector panel (`Cmd+I`) shows tags (editable), favorite toggle, archive toggle, dates
7. User edits freely — all changes auto-save; no explicit Save button
8. User closes window → changes are persisted

Edge cases:

- No unsaved-changes warning when closing edit window
- If the same prompt is open in both edit window and previewed in the main window, changes sync in real-time
- Removing `{{variable}}` from content removes its corresponding variable record and stored value — silent data loss of the fill value
- Adding a tag that already exists (case-insensitive) does not create a duplicate
- Archive toggle in inspector removes prompt from "All Prompts" and "Recently Used" views — no undo
- Attachment deletion is immediate with no confirmation dialog

Flow 7 — Quick Copy via Hotkey Launcher**Surface:** Hotkey Launcher**Trigger:** `Cmd+Shift+P` from any application**Steps:**

1. Floating HUD panel appears (500px wide) over the frontmost app

2. Search field is auto-focused
3. Default view: up to 9 prompts shown (favorites prioritised, then by recency)
4. Each row shows: Cmd+1–9 shortcut badge, title, star (if favorited), tags
5. User types to filter, or uses Up/Down arrow keys to select, or presses Cmd+1–9
- 6a. If `defaultAction = "copy"`: prompt is copied immediately, launcher dismisses
- 6b. If `defaultAction = "showOptions"` (default): action view appears with "Copy" button and Cmd+C shortcut
7. User presses Cmd+C or clicks "Copy" → clipboard populated, launcher dismisses

Edge cases:

- No variable-filling UI in launcher — values used are whatever was last saved in `SwiftData`
- Pressing Esc from action view returns to search view (not close)
- Pressing Esc from search view closes launcher
- Launcher does not prevent interaction with the app underneath
- If no prompts exist: "No prompts yet" centered message; launcher is still functional but empty
- If search returns no results: "No results" message
- `lastUsedAt` is updated on copy, feeding "Recently Used" in main window

Flow 8 — Search

Surface: All three surfaces

Trigger: User types in any search field

Steps (Main Window):

1. User types in sidebar search field
2. List filters in real-time across title, content, and tag names (case-insensitive contains)
3. Active filter (All/Recent/Archived/Tag) is applied in addition to search
4. Clear button (×) appears when search is non-empty; click to reset
5. Empty state shown if no matches: "No matching prompts" with magnifying glass icon

Steps (Menu Bar):

1. User types in search field at top of popover
2. Active filter tab (Favorites/All/Archived) combined with search
3. Same real-time filtering behaviour

Steps (Hotkey Launcher):

1. User types in auto-focused search field
2. Results filtered from all non-archived prompts
3. Max 9 results displayed

Edge cases:

- Search in main window does not cross filter boundaries — e.g. searching while in "Archived" only searches archived prompts
- No indication that search is scoped to the active filter
- Tag name search works, but the matched tag is not highlighted in the result row
- Search state is not preserved when switching filters in the main window sidebar
- No search history or recent queries

Flow 9 — Manage Tags**Surface:** Main Window (Inspector / New Prompt / Edit Prompt)**Trigger:** Click "+" in tag section of Inspector or New Prompt view**Steps:**

1. TagPickerPopover opens (260×380)
2. Top section: existing unselected tags shown as pills — click to add
3. Divider (only if existing tags available)
4. Bottom section: "Create New" — type name, select color (6 swatches), click "Create"
5. New tag is created and immediately applied to the prompt
6. Popover remains open for adding more tags
7. Click outside popover to dismiss

Remove tag:

1. Tag pills shown in inspector with hover-reveal × button
2. Click × → tag removed from prompt (not deleted from library)

Edge cases:

- No way to rename or delete a tag globally from within the app
- Tag name is lowercased automatically before storage
- Creating a tag with the same name (case-insensitive) as an existing tag reuses the existing tag
- No feedback when a duplicate name is detected — the new tag just silently maps to the existing one
- Tags with long names truncate in pills — no tooltip showing full name
- Tag color cannot be changed after creation

Flow 10 — Favorite & Archive**Surface:** All surfaces**Trigger:** Star icon (favorite) or archive action**Favorite:**

1. Click star in Inspector (edit/preview) → prompt marked as favorite; star fills yellow

2. In menu bar: ellipsis menu → "Favorite" / "Unfavorite"
3. In sidebar context menu: "Favorite" / "Unfavorite"
4. Favorited prompts appear in "★ Favorites" filter on all surfaces

Archive:

1. In Inspector: click "Archive" button → prompt removed from All/Recent, appears in Archived filter
2. In sidebar context menu: "Archive" / "Unarchive"
3. In menu bar: ellipsis menu → "Archive" / "Unarchive"

Edge cases:

- Archiving a prompt does not unfavorite it — an archived-but-favorited prompt disappears from "★ Favorites" (archived are excluded from all non-archive filters)
- No undo for archive (no Cmd+Z, no toast with "Undo")
- "Recently Used" filter excludes archived prompts — archiving removes a prompt from recents even if recently copied

Flow 11 — Delete a Prompt

Surface: Main Window (primary), Menu Bar

Trigger: Delete key (main window), context menu "Delete", or menu bar ellipsis "Delete"

Steps (Main Window):

1. Prompt selected in sidebar
2. User presses Delete key or selects "Delete" from context menu
3. Alert: "Are you sure you want to delete '[title]'? This cannot be undone."
4. Buttons: "Delete" (destructive red) / "Cancel"
5. On confirm: prompt deleted from SwiftData, sidebar selection cleared

Steps (Menu Bar):

1. Prompt row expanded
2. User selects "Delete" from ellipsis menu
3. Alert appears (same wording)
4. On confirm: prompt deleted, row removed from list

Edge cases:

- No soft delete / trash — deletion is permanent
- Related template variables and attachments are cascade-deleted (attachment files also removed from disk)
- Tags are NOT deleted — they remain in the library as orphaned tags
- No undo for deletion
- Deleting a prompt that is currently open in an Edit window leaves the edit window open showing "Prompt Not Found"

Flow 12 — Export & Import Data

Surface: Settings → Data tab

Trigger: Click "Export All Prompts..." or "Import Prompts..."

Export:

1. Click "Export All Prompts..."
2. NSSavePanel opens, default filename `pault-prompts.json`
3. User chooses location, clicks Save
4. JSON file written — contains all prompts, tags, template variables (not attachments)
5. No confirmation or success toast after export

Import:

1. Click "Import Prompts..."
2. NSOpenPanel opens, filtered to `.json` files
3. User selects file, clicks Open
4. Prompts decoded; duplicates (matching UUID) are skipped
5. Alert shown: "Imported X prompt(s)." or "No new prompts to import (all already exist)."

Edge cases:

- Attachments are NOT included in export — no warning communicates this
- Rich text (RTFD `attributedString`) is NOT included in export — only plain content — no warning
- No export format version checking on import (version field exists but is not validated)
- No progress indicator for large libraries
- Tags are recreated by name on import; tag colors are NOT preserved (new tags get the `resolveTag` default, which inserts with no color specified)
- If the JSON file is malformed, import silently fails (error logged, no user alert)

Flow 13 — Change Settings

Surface: Settings (Cmd+,)

Trigger: Cmd+, keyboard shortcut, "Settings" button in menu bar, or right-click menu bar → "Preferences..."

General tab:

- Launch at login toggle → immediate `SMAccessService` registration
- Show dock icon toggle → immediate `NSApp` policy change
- Default action picker → stored in `AppStorage`, read by Hotkey Launcher

Hotkey tab:

- Displays "Cmd+Shift+P" read-only badge

- No interaction possible

Appearance tab:

- Font size (Small/Medium/Large) → stored in AppStorage; **not read by any view** — no effect
- Compact mode → stored in AppStorage; **not read by any view** — no effect
- Accent color → stored in AppStorage; **not read by any view** — no effect

Data tab:

- See Flow 12

Edge cases:

- Three Appearance settings (font size, compact mode, accent color) are persisted but have no effect on the UI — they are dead controls
- No visual confirmation that General tab toggles succeeded (except dock icon appearing/disappearing)
- Settings window is 450×320; on very small displays (rare for macOS) content may clip

Flow 14 — Menu Bar Context Menu

Surface: Menu Bar (right-click)

Trigger: Right-click on Pault status bar icon

Steps:

1. Right-click menu bar icon → NSMenu appears with 5 items
2. "Open Pault" → activates app, brings main window to front
3. "Preferences..." → opens Settings window (Cmd+,)
4. "About Pault" → opens About window
5. (separator)
6. "Quit Pault" → terminates app (saves SwiftData context first)

Left-click: Opens popover (unchanged)

Edge cases:

- "Open Pault" uses a window-lookup heuristic (`!isKind(of: NSPanel.self)`) — if main window is closed (not just hidden), this may activate the app but not show a window
- About window opened via notification bridge (AppDelegate → NotificationCenter → SwiftUI openWindow) — small timing risk if ContentView has not appeared yet
- "Preferences..." uses `Selector(("showSettingsWindow:"))` — private AppKit API, could break in future macOS versions

Flow 15 — Attachments

Surface: Main Window (Edit Prompt)

Trigger: "+" button in Attachments strip, or drag files onto the RichTextEditor / attachment strip

Add via button:

1. Click "+" in AttachmentsStripView
2. NSOpenPanel opens (multi-select, filtered to images, video, audio, PDF, Office formats)
3. User selects files, clicks Open
4. Files stored via AttachmentManager; thumbnails generated
5. Thumbnails appear in horizontal strip

Add via drag-drop:

1. User drags file(s) onto the editor or strip
2. Strip border highlights in accent color
3. On drop: files processed same as button add

Interact with attachment:

- Context menu: "Open" (in default app), "Quick Look", "Insert Inline" (images only), "Delete"
- "Insert Inline" posts notification → RichTextEditor embeds image in text body

Edge cases:

- Deleting an attachment is immediate with no confirmation dialog (inconsistent with prompt deletion)
- Attachments are NOT exported in JSON export (Flow 12) — no warning at point of attachment creation
- No attachment size limit enforced in UI — large files will be stored without warning
- If AttachmentManager fails to generate a thumbnail, fallback icon shown silently
- Read-only preview (PromptPreviewView) shows attachments with Open/Quick Look only — no Insert Inline

Section 2: Heuristic Audit

Evaluated against Nielsen's 10 Usability Heuristics and macOS Human Interface Guidelines (HIG). Issues are grouped by heuristic, then ordered by severity within each group.

Summary Table

#	Severity	Surface	Heuristic	Issue
---	----------	---------	-----------	-------

---	-----	-----	-----	-----
-----	-------	-------	-------	-------

H-01	Critical	Settings → Appearance	Visibility of System Status	Font size, compact mode, and accent color controls have no effect — settings are stored but never read
------	-----------------	-----------------------	-----------------------------	--

H-02	Critical	Export (Data tab)	Error Prevention	Attachments and rich text are silently excluded from JSON export
------	-----------------	-------------------	------------------	--

H-03	Major	Hotkey Launcher	Flexibility & Efficiency	No variable-filling UI in launcher; copied output may contain unresolved <code>{{variables}}</code> with no warning
------	--------------	-----------------	--------------------------	---

H-04	Major	Onboarding	Match Between System & World	Page 3 references "paste instantly" — paste was removed in Phase 2.5
------	--------------	------------	------------------------------	--

| H-05 | **Major** | Main Window | Recognition Over Recall | `{{variable}}` syntax is never explained in the UI; users must discover it by trial or locate the PDF guide |

| H-06 | **Major** | All surfaces | Visibility of System Status | No success feedback after Export (no toast, no alert, no confirmation) |

| H-07 | **Major** | Edit Window | Error Prevention | Closing an edit window with unsaved in-progress text shows no warning (though auto-save mitigates this, the debounce window is a risk) |

| H-08 | **Major** | Edit Window | User Control & Freedom | Removing `{{variable}}` from content silently deletes the stored fill value — no undo, no warning |

| H-09 | **Major** | Menu Bar right-click | Consistency & Standards | "Preferences..." uses a private AppKit selector (`showSettingsWindow:`) — fragile; could silently break on macOS update |

| H-10 | **Major** | All surfaces | User Control & Freedom | No undo (`Cmd+Z`) for archive, favorite-toggle, or tag removal — all are instantaneous and irreversible at the app level |

| H-11 | **Minor** | Settings → Data | Consistency & Standards | "Clear All Data" uses `confirmationDialog` (sheet); prompt deletion uses `Alert` — inconsistent patterns for equivalent destructive actions |

| H-12 | **Minor** | Import (Data tab) | Error Prevention | Malformed JSON import fails silently — no user-facing error alert |

| H-13 | **Minor** | Import (Data tab) | Error Prevention | Tag colors not preserved on import — imported tags receive no color (default gray), but no warning is shown |

| H-14 | **Minor** | Sidebar | Visibility of System Status | Search is silently scoped to the active filter — no label or indicator communicates "searching within Archived" etc. |

| H-15 | **Minor** | Sidebar | Recognition Over Recall | Double-clicking a prompt to open the edit window is undiscovered affordance — no visual hint or tooltip |

| H-16 | **Minor** | Sidebar | Visibility of System Status | "Recently Used" empty state gives no hint that copying a prompt populates this view |

| H-17 | **Minor** | New Prompt (main) | User Control & Freedom | Closing the New Prompt window via `Cmd+W` discards content with no confirmation |

| H-18 | **Minor** | Attachments | Consistency & Standards | Deleting an attachment has no confirmation dialog; deleting a prompt does — inconsistent destructive action patterns |

| H-19 | **Minor** | Tag management | User Control & Freedom | No way to rename or delete a tag globally; orphaned tags accumulate silently after prompt deletion |

| H-20 | **Minor** | Menu bar → Open Pault | Reliability (HIG) | "Open Pault" uses a window-class heuristic to find the main window; if the window was closed (not just hidden), the app activates but no window appears |

| H-21 | **Minor** | Onboarding | Match Between System & World | No way to re-view onboarding after first launch — no "Show Welcome Guide" in Help menu or Settings |

| H-22 | **Minor** | About window | Consistency & Standards (HIG) | About window opened via Notification bridge from `AppDelegate` — minor timing risk if main `ContentView` has not yet appeared |

H-01 — Appearance Settings Have No Effect

Severity: Critical · **Surface:** Settings → Appearance tab

Heuristic: Visibility of System Status

`fontSizePreference`, `useCompactMode`, and `accentColorPreference` are written to `@AppStorage` in `PreferencesView.swift` but are not read by any other view in the codebase. Changing these settings produces no visible change in the application. Users who discover these controls and interact with them receive implicit feedback that the app responded (the swatch highlights, the toggle flips) but the promised change never materialises.

Recommended fix: Either wire the values to the UI (read `@AppStorage` in `ContentView`, `SidebarView`, and `MenuBarContentView` and apply `.tint()`, row padding, and font scaling), or remove the controls until they are implemented. Dead controls that appear functional are more damaging than absent controls.

H-02 — Export Silently Excludes Attachments and Rich Text

Severity: Critical · **Surface:** Settings → Data tab → Export

Heuristic: Error Prevention

`ExportService.exportAll()` serialises only plain content (`String`) and template variable `defaultValues`. It does not include:

- `attributedContent` (RTFD rich text, including inline images)
- Attachment records or their files

A user who relies on export as a backup, then clears their library (Flow 12 allows "Clear All Data" immediately below the Export button), will lose all rich text formatting and all file attachments permanently. There is no warning at the point of export or at the point of "Clear All Data".

Recommended fix (short term): Add a callout in the Data tab below the Export button: "Note: Attachments and rich text formatting are not included in the JSON export." Add the same note to the Clear All Data confirmation dialog. Long term: include RTFD as a base64 field in the export bundle.

H-03 — Hotkey Launcher Copies Unresolved Variables Without Warning

Severity: Major · **Surface:** Hotkey Launcher

Heuristic: Flexibility & Efficiency / Error Prevention

The launcher is the fastest path to copy a prompt, but it has no variable-filling UI. If a prompt contains `{{variables}}` with no stored values, the copied output contains literal `{{variable_name}}` text. The user's target application (an AI chat, an email, a document) receives the template syntax rather than the intended content. The launcher shows no indicator that variables are unfilled, and gives no warning before or after copying.

Recommended fix: Add a small indicator in each launcher result row when a prompt has unfilled variables (e.g. a { } badge or secondary text "Has unfilled variables"). Optionally, if `defaultAction = "showOptions"`, show a brief variable-fill form in the action view before copying.

H-04 — Onboarding References Removed Feature

Severity: Major · **Surface:** Onboarding (Page 3)

Heuristic: Match Between System & World

Page 3 ("Built for Speed") states: *"Press Cmd+Shift+P to search, Cmd+1-9 to select, and Return to copy or paste instantly."* The paste feature was removed in Phase 2.5. The app no longer pastes automatically — it copies to clipboard only, and the user must paste manually. New users who read the onboarding will expect behaviour that no longer exists.

Recommended fix: Update page 3 description to: "Press Cmd+Shift+P to search, Cmd+1–9 to select, and Return to copy to your clipboard instantly."

H-05 — Template Variable Syntax Is Never Explained In-App

Severity: Major · **Surface:** Main Window / New Prompt

Heuristic: Recognition Over Recall

The `{{variable}}` syntax is the app's most distinctive feature, but it is nowhere documented in the UI itself. The New Prompt creation window shows a "Template Variables" section that appears dynamically as the user types, but there is no prompt, placeholder, or hint telling the user what causes it to appear. A user who does not read the PDF user guide has no path to discovery other than accident.

Recommended fix: Add a single help line below the content field in `NewPromptView` and `PromptDetailView`: *"Type `{{variable}}` to create a template placeholder."* A help button (?) linking to the user guide would also serve this. Inline empty-state text on the Variables section (*"Type `{{name}}` anywhere in your content to add a variable"*) would catch users who are already in the right area.

H-06 — Export Gives No Success Feedback

Severity: Major · **Surface:** Settings → Data tab

Heuristic: Visibility of System Status

After the user selects a save location and the export completes, nothing in the UI changes. There is no toast, no alert, no status message. The button simply returns to its resting state. The user has no confirmation that the file was written successfully (or that anything happened at all, if they dismissed the panel quickly).

Recommended fix: Show a brief inline success message (*"Exported 42 prompts to pault-prompts.json"*) below the Export button, or use a non-modal toast consistent with the existing copy toast pattern. Surface errors too — if the write fails, log an alert.

H-07 — Edit Window Has No Close Warning

Severity: Major · **Surface:** Edit Window

Heuristic: Error Prevention

Auto-save debounces at 500ms. If a user types content and closes the edit window (Cmd+W or red button) within that 500ms window, the final edits are lost with no warning. While this is a narrow race condition in practice, it is a silent data-loss path. Additionally, users coming from other macOS text editors (Notes, TextEdit) expect a "Save changes?" dialog when closing a window with unsaved content.

Recommended fix: On `windowWillClose`, flush any pending debounced saves synchronously before the window closes. This eliminates the race condition without adding a dialog.

H-08 — Removing a Variable From Content Silently Deletes Its Fill Value

Severity: Major · **Surface:** Edit Window

Heuristic: User Control & Freedom

When the user edits prompt content and removes a `{{variable}}` token, `TemplateEngine.syncVariables()` is called and the corresponding `TemplateVariable` record (including its stored `defaultValue`) is cascade-deleted from `SwiftData`. If the user accidentally removes a variable and re-adds it, the fill value is gone. There is no undo path.

Recommended fix: Before deleting a `TemplateVariable` during sync, check if `defaultValue` is non-empty. If so, preserve the record as an "orphaned variable" temporarily (e.g. flag it), giving the user a chance to undo the content edit before the value is purged. Alternatively, implement Cmd+Z undo for content edits that trigger variable sync.

H-09 — Private AppKit Selector for Preferences

Severity: Major · **Surface:** Menu Bar right-click context menu

Heuristic: Consistency & Standards

`NSMenuItem(title: "Preferences...", action: Selector(("showSettingsWindow:")))` uses an undocumented AppKit method. This is the same pattern used internally by SwiftUI's Settings scene, but it is not a public API. Apple has broken this pattern before. If it silently stops working in a future macOS release, the menu item will appear but do nothing — no crash, no error, just a broken menu item.

Recommended fix: Post a `Notification.Name.openSettingsWindow` notification (same bridge pattern used for the About window) and handle it in `PaultApp.swift` via `openSettings` environment action, or use `NSApp.sendAction(Selector(("showPreferencesWindow:")), ...)` as a fallback. Alternatively, expose a `@Environment(\.openSettings)` handler (available from macOS 14) in a SwiftUI wrapper.

H-10 — No Undo for Reversible Destructive Actions

Severity: Major · **Surface:** All surfaces

Heuristic: User Control & Freedom

The following actions are instantaneous and irreversible with no undo:

- Toggling archive state
- Toggling favorite state
- Removing a tag from a prompt
- Clearing all variable values ("Clear" button in TemplateVariablesView)

None of these show a confirmation dialog (which would be excessive for lightweight actions), but none offer a toast-based "Undo" affordance either. This is a missed expectation for macOS users who rely on Cmd+Z.

Recommended fix: Implement NSUndoManager integration for SwiftData mutations, or at minimum provide a brief "Undo" action in the copy toast style for the archive toggle (the highest-impact case, since archiving removes a prompt from normal view).

H-11 — Inconsistent Destructive Action Patterns

Severity: Minor · **Surface:** Settings → Data tab vs. prompt deletion

Heuristic: Consistency & Standards

"Clear All Data" uses `confirmationDialog` (sheet-style, anchored to the button). Prompt deletion uses `Alert` (centred modal). Both are destructive, irreversible actions. Using different patterns for equivalent operations creates a subtle inconsistency that undermines user confidence.

Recommended fix: Standardise on `Alert` for all destructive confirmations (consistent with the prompt deletion pattern and macOS HIG guidance on destructive alerts).

H-12 — Import Failure Is Silent

Severity: Minor · **Surface:** Settings → Data tab → Import

Heuristic: Error Prevention / Visibility of System Status

If the selected JSON file is malformed or the wrong format, `ExportService.importPrompts()` catches the `DecodingError` and returns `nil`. The calling code in `DataTab` only shows an alert if the return value is non-`nil` — so a decode failure produces no feedback to the user. The file picker simply closes and nothing happens.

Recommended fix: Return a `Result<Int, Error>` from `importPrompts()` instead of `Int?`, and show an error alert ("Could not read this file. Make sure it is a valid Pault export.") when the result is a failure.

H-13 — Tag Colors Not Preserved on Import

Severity: Minor · **Surface:** Import

Heuristic: Error Prevention

`ExportService.exportAll()` serialises tags as an array of name strings only — color is not included in `PromptExportRecord`. On import, `resolveTag(named:in:)` creates new `Tag` records using

`Tag(name : )` with no `color` parameter. All imported tags will have whatever default color `Tag.init` assigns (none, rendering as gray). Users who carefully color-coded their tag library will lose that organisation after an export/import round-trip.

Recommended fix: Add a `tagColor` field to `PromptExportRecord` and serialise `tag.color` during export. On import, pass the color to `Tag(name : color : )`.

H-14 — Search Scope Is Not Communicated

Severity: Minor · **Surface:** Main Window sidebar

Heuristic: Visibility of System Status

When the user types in the search field while the "Archived" filter is active, results are scoped to archived prompts only. There is no label, badge, or visual indicator communicating "Searching in: Archived". A user who switches filters while searching may be confused by changing result counts without understanding why.

Recommended fix: Add a small secondary label below the search field when a non-default filter is active: "Searching in: Archived" or "Searching in: #tagname". This is a single `Text` view conditioned on `selectedFilter`.

H-15 — Double-Click to Edit Is an Undiscovered Affordance

Severity: Minor · **Surface:** Main Window sidebar

Heuristic: Recognition Over Recall

Double-clicking a sidebar row opens the Edit window. This is the primary path to full editing, but there is no visual affordance, hover state, tooltip, or documentation in the UI that reveals this. Users who single-click to preview and never try double-clicking will not know the Edit window exists unless they discover the toolbar "Edit" button or `Cmd+E` shortcut.

Recommended fix: Add a tooltip on prompt rows ("Double-click to edit") or add a visible "Edit" button that appears on row hover. The toolbar button already exists — ensuring its discoverability is the most important fix.

H-16 — "Recently Used" Empty State Is Uninformative

Severity: Minor · **Surface:** Main Window sidebar

Heuristic: Visibility of System Status

When the "Recently Used" filter is active and no prompts have been copied, the empty state shows "No recently used prompts" with an icon — but gives no guidance on how to populate it. A new user has no way to know that copying a prompt (`Cmd+C`) is what triggers `lastUsedAt` and feeds this view.

Recommended fix: Update the empty state help text to: "Copy a prompt to see it here." This matches the existing pattern used for the "All Prompts" empty state hint ("Press `Cmd+N` to create one").

H-17 — New Prompt Window Discards Content Without Warning

Severity: Minor · **Surface:** New Prompt window

Heuristic: User Control & Freedom

Pressing Cmd+W or the red close button on the New Prompt window discards the title and content the user was writing, with no confirmation. The Cancel button (Esc / Cmd+.) is intentionally destructive, but the window close gesture is more ambiguous — on macOS, closing a document window typically triggers a save prompt.

Recommended fix: Add a `windowShouldClose` check: if either field is non-empty, show an alert ("Discard this prompt?" / "Discard" / "Cancel"). This matches the macOS convention for unsaved document windows.

H-18 — Attachment Deletion Has No Confirmation

Severity: Minor · **Surface:** Edit Window → Attachments strip

Heuristic: Consistency & Standards

Deleting an attachment (via context menu → "Delete") is immediate and permanent — the file is removed from disk and the record deleted from SwiftData. There is no confirmation dialog, unlike prompt deletion which always shows an alert. This inconsistency is especially notable because attachment deletion is file-system destructive (the embedded file may exist nowhere else).

Recommended fix: Add a confirmation alert before deleting an attachment: "Remove '[filename]'? This cannot be undone."

H-19 — Orphaned Tags Accumulate Silently

Severity: Minor · **Surface:** Tag management

Heuristic: User Control & Freedom

When a prompt is deleted, its tag relationships are removed (nullify delete rule), but the Tag records themselves persist. Over time, a library with many deleted prompts will accumulate tags that are attached to no prompts. These appear in the TagPickerPopover as valid options, creating noise. There is no way to delete, rename, or audit tags from within the app.

Recommended fix: Add a "Manage Tags" section to the Data tab in Settings, listing all tags with their prompt count. Tags with 0 prompts could be shown with a warning indicator and a delete button.

H-20 — "Open Pault" May Activate Without Showing a Window

Severity: Minor · **Surface:** Menu Bar right-click → "Open Pault"

Heuristic: Reliability (macOS HIG)

`openMainWindow()` in `AppDelegate` uses `NSApp.windows.first(where: { $0.identifier?.rawValue == "main" || ... })` to find the main window. If the user has closed

the main window entirely (not just minimised), `NSApp.windows` may not contain it (SwiftUI's `WindowGroup` does not keep a persistent `NSWindow` reference after close). In this case, `NSApp.activate()` runs but no window is brought forward — the app is in the foreground with no visible UI.

Recommended fix: After `activate`, check if a matching window was found. If not, post `Notification.Name.createNewPrompt` (or a similar notification) to trigger SwiftUI to open a new main window instance, which is idiomatic for `WindowGroup`-based apps.

H-21 — No Way to Re-View Onboarding

Severity: Minor · **Surface:** Global

Heuristic: Match Between System & World

Once `hasCompletedOnboarding` is set to `true`, there is no way to re-open the onboarding flow from within the app. macOS convention (and user expectation) is that a "Welcome" or "Getting Started" item appears in the Help menu. This is particularly useful for users who rush through onboarding on first launch.

Recommended fix: Add a "Show Welcome Guide..." item to the macOS Help menu (`CommandGroup(after: .help)`). It would reset `hasCompletedOnboarding = false` and trigger the sheet on the next main window appearance, or open it immediately via a sheet binding in `ContentView`.

H-22 — About Window Uses Notification Bridge With Timing Risk

Severity: Minor · **Surface:** About Window

Heuristic: Reliability

`AppDelegate.openAboutWindow()` posts `Notification.Name.openAboutWindow`, which is handled by `.onReceive` in `ContentView`'s body. If the main `ContentView` has not yet appeared when the notification fires (e.g. on first launch before the window renders), the notification will be missed and the About window will not open.

Recommended fix: Use `NotificationCenter.default.publisher` with a small dispatch queue delay, or store a pending "open about" flag in `AppDelegate` and check it in `ContentView.onAppear`. The same bridge is used for the `createNewPrompt` flow — audit that handler for the same timing risk.

End of UX Audit — Pault Phase 2.5